

Expert Systems – Introduction

- **Expert Systems** are computer programs that exhibit intelligent behavior.
- They are concerned with the concepts and methods of symbolic inference, or reasoning, by a computer, and how the knowledge used to make those inferences will be represented.
- Achieving expert-level competence in solving problems in task areas by bringing to bear a body of knowledge about specific tasks is called *knowledge-based* or *expert system*.
- The term expert system is reserved for programs whose knowledge base contains the knowledge used by human experts.

- Expert systems and knowledge-based systems are used synonymously.
- The area of human intellectual endeavor to be captured in an expert system is called the *task domain*.
- *Task* refers to some goal-oriented, problem-solving activity.
- *Domain* refers to the area within which the task is being performed. Typical tasks are diagnosis, planning, scheduling, configuration and design.

- Building an expert system is known as *knowledge engineering* and its practitioners are called *knowledge engineers*.
- The knowledge engineer must make sure that the computer has all the knowledge needed to solve a problem.
- The knowledge engineer must choose one or more forms in which to represent the required knowledge as symbol patterns in the memory of the computer -- that is, he (or she) must choose a *knowledge representation*.
- He must also ensure that the computer can use the knowledge efficiently by selecting from a handful of *reasoning methods*

Building Blocks of Expert Systems

- Every expert system consists of two principal parts: the knowledge base; and the reasoning, or inference, engine.
- The *knowledge base* of expert systems contains both factual and heuristic knowledge.
- *Factual knowledge* is that knowledge of the task domain that is widely shared, typically found in textbooks or journals, and commonly agreed upon by those knowledgeable in the particular field.
- *Heuristic knowledge* is the less rigorous, more experiential, more judgmental knowledge of performance.
- In contrast to factual knowledge, heuristic knowledge is rarely discussed, and is largely individualistic. It is the knowledge of good practice, good judgment, and plausible reasoning in the field. It is the knowledge that underlies the "art of good guessing."

Building Blocks of Expert Systems

- *Knowledge representation* formalizes and organizes the knowledge. One widely used representation is the *production rule*, or simply *rule*. A rule consists of an IF part and a THEN part (also called a *condition* and an *action*).
- The IF part lists a set of conditions in some logical combination.
- The piece of knowledge represented by the production rule is relevant to the line of reasoning being developed if the IF part of the rule is satisfied; consequently, the THEN part can be concluded, or its problem-solving action taken.
- Expert systems whose knowledge is represented in rule form are called *rule-based systems*.

Building Blocks of Expert Systems

- Another widely used representation, called the unit (also known as frame, schema, or list structure) is based upon a more passive view of knowledge.
- The unit is an assemblage of associated symbolic knowledge about an entity to be represented. Typically, a unit consists of a list of properties of the entity and associated values for those properties.
- Since every task domain consists of many entities that stand in various relations, the properties can also be used to specify relations, and the values of these properties are the names of other units that are linked according to the relations.
- One unit can also represent knowledge that is a "special case" of another unit, or some units can be "parts of" another unit.

Building Blocks of Expert Systems

- The problem-solving model, or paradigm, organizes and controls the steps taken to solve the problem.
- One common but powerful paradigm involves chaining of IF-THEN rules to form a line of reasoning. If the chaining starts from a set of conditions and moves toward some conclusion, the method is called forward chaining.
- If the conclusion is known (for example, a goal to be achieved) but the path to that conclusion is not known, then reasoning backwards is called for, and the method is backward chaining.
- These problem-solving methods are built into program modules called inference engines or inference procedures that manipulate and use knowledge in the knowledge base to form a line of reasoning.

Building Blocks of Expert Systems

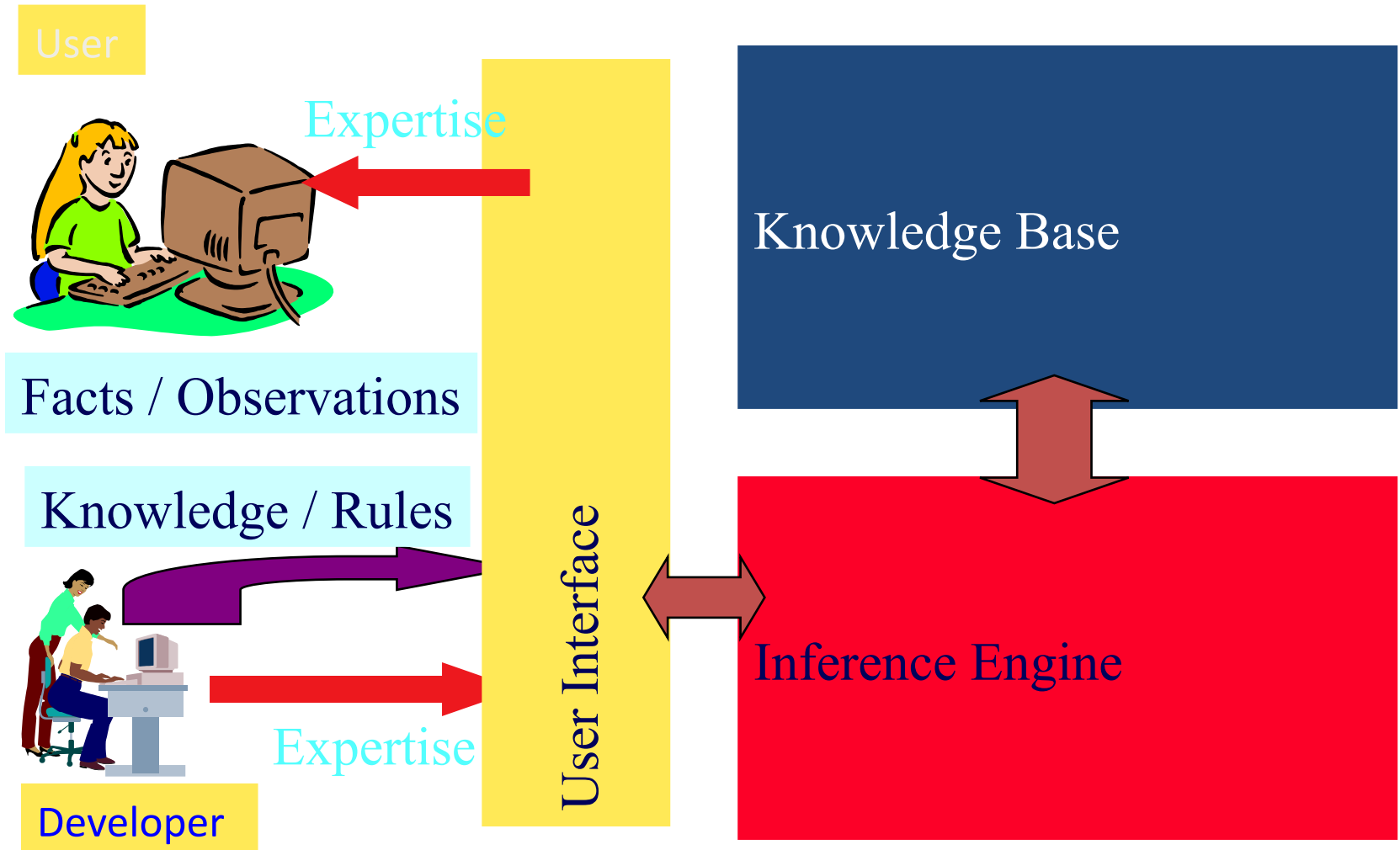
- The *knowledge base* an expert uses is what he learned at school, from colleagues, and from years of experience. Presumably the more experience he has, the larger his store of knowledge.
- Knowledge allows him to interpret the information in his databases to advantage in diagnosis, design, and analysis.
- Though an expert system consists primarily of a knowledge base and an inference engine, a couple of other features are worth mentioning: reasoning with uncertainty, and explanation of the line of reasoning.

Building Blocks of Expert Systems

- Knowledge is almost always incomplete and uncertain. Thus a rule may have associated with it a *confidence factor* or a weight.
- The set of methods for using uncertain knowledge in combination with uncertain data in reasoning is called *reasoning with uncertainty*.
- A subclass of methods for reasoning with uncertainty is called "fuzzy logic," and the systems are known as "fuzzy systems."
- Because an expert system uses uncertain or heuristic knowledge (as humans do) its credibility is often in question (as with humans). When an answer to a problem is questionable, we tend to want to know the rationale.
- If the rationale seems plausible, we tend to believe the answer. So it is with expert systems.
- Most expert systems have the ability to answer questions of the form: "Why is the answer X?" Explanations can be generated by tracing the line of reasoning used by the inference engine.

Building Blocks of Expert Systems

- The most important ingredient in any expert system is knowledge. The power of expert systems resides in the specific, high-quality knowledge they contain about task domains.
- Researchers will continue to explore and add to the current repertoire of knowledge representation and reasoning methods. But in knowledge resides the power.
- Because of the importance of knowledge in expert systems and because the current knowledge acquisition method is slow and tedious, much of the future of expert systems depends on breaking the knowledge acquisition bottleneck and in codifying and representing a large knowledge infrastructure.



Components of Expert Systems

knowledge base

- contains essential information about the problem domain
- often represented as **facts** and **rules**

inference engine

- mechanism to derive new knowledge from the knowledge base and the information provided by the user
- often based on the **use of rules**

user interface

- interaction with end users
- development and maintenance of the knowledge base

Concepts & Characteristics of Expert Systems

knowledge acquisition

- transfer of knowledge from humans to computers
- sometimes knowledge can be acquired directly from the environment
 - machine learning, neural networks

knowledge representation

- suitable for storing and processing knowledge in computers

inference

- mechanism that allows the generation of new conclusions from existing knowledge in a computer

explanation

- illustrates to the user how and why a particular solution was generated

- Rules can be used to formulate a theory of human information processing (Newell & Simon)
 - rules are stored in long-term memory
 - temporary knowledge is kept in short-term memory
 - (external) sensory input triggers the activation of rules
 - activated rules may trigger further activation (internal input; “thinking”)
 - a cognitive processor combines evidence from currently active rules
- this model is the basis for the design of many rule-based systems
(*production systems*)

Keys to Expert Systems Success

- convincing ideas
 - rules, cognitive models
- practical applications
 - medicine, computer technology, ...
- separation of knowledge and inference
 - expert system *shell*
 - allows the re-use of the “machinery” for different domains
- concentration on domain knowledge
 - general reasoning is too complicated

When not to use an Expert System

- Expert systems are not suitable for all types of domains and tasks

They are not useful or preferable, when ...

- efficient conventional algorithms are known
- the main challenge is computation, not knowledge
- knowledge cannot be captured efficiently or used effectively
- users are reluctant to apply an expert system, e.g. due to criticality of task, high risk or high security demands

- knowledge base
- inference engine
- working memory
- agenda
- explanation facility
- knowledge acquisition facility
- user interface

Architecture of Expert Systems

Knowledge-Base / Rule-Base

store expert knowledge as condition-action-rules
(aka: if-then- or premise-consequence-rules)

Working Memory

stores initial facts and generated facts derived by inference engine; maybe with additional parameters like the “degree of trust” into the truth of a fact $\cong$ certainty factor

Architecture of Expert Systems

Inference Engine

- matches **condition-part** of rules against facts stored in Working Memory (**pattern matching**);
- rules with satisfied condition are **active rules** and are placed on the **agenda**;
- among the active rules on the agenda, one is **selected** (see **conflict resolution, priorities of rules**) as next rule for
- **execution** (“firing”) – consequence of rule is added as new fact(s) to Working Memory

Inference Engine + additional components might be necessary for other functions, like

- calculation of **certainty values**,
- determining **priorities** of rules,
- **conflict resolution** mechanisms,
- a **truth maintenance system (TMS)** if reasoning with **defaults** and **beliefs** is requested

IF ... THEN Rules

Rule: Red_Light

IF the light is red (antecedent)

THEN stop (consequent)

Rule: Green_Light

IF the light is green

THEN go

Production Rules

the light is red ==> stop (left-hand side - antecedent)
(right-hand side - consequent)

the light is green ==> go